

Coding & Solution

Date	26 June 2026
Team ID	SWTID-2026-8613
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

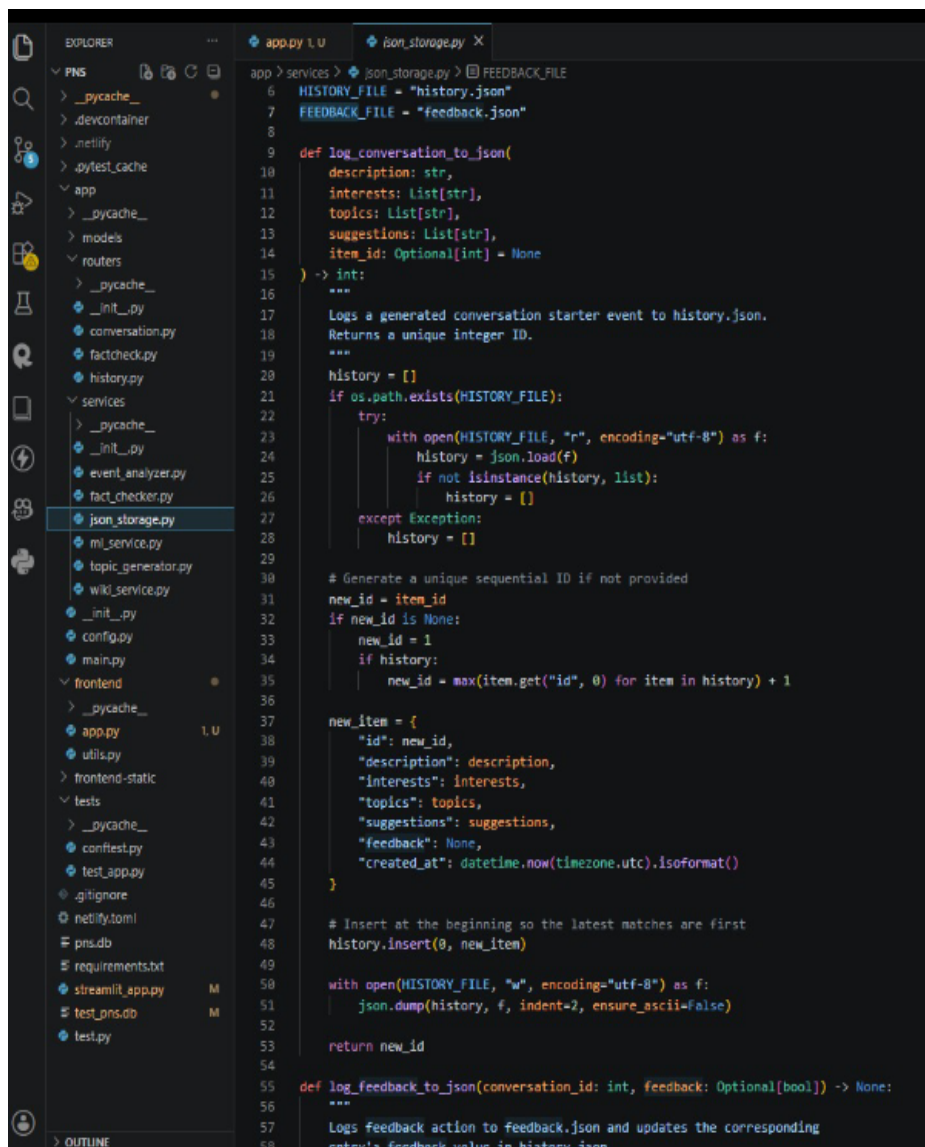
Field	Details
Repository Link / URL	https://github.com/devanshumeena1/Personalized-Networking-Assistant-SWTID-2026-8613
Programming Language(s)	Python
Framework(s) Used	FastAPI, Streamlit, Hugging Face Transformers
Key Features Implemented	Feedback Logger Service, Conversation History View, Feedback History View
Pending / Incomplete Features	None
Setup / Run Instructions	<pre> pip install -r requirements.txt uvicorn main:app --reload streamlit run app.py </pre>

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Feedback Logger Service

Description : The Feedback Logger Service is responsible for recording user feedback on the generated conversation suggestions. It stores the suggestion text, the user's feedback (Like/Dislike), and the corresponding timestamp in a local JSON file. This structured storage enables future analysis of user preferences and helps improve the quality of conversation recommendations. The service is designed with modular functions for writing and retrieving feedback data while ensuring clean and maintainable code.



```

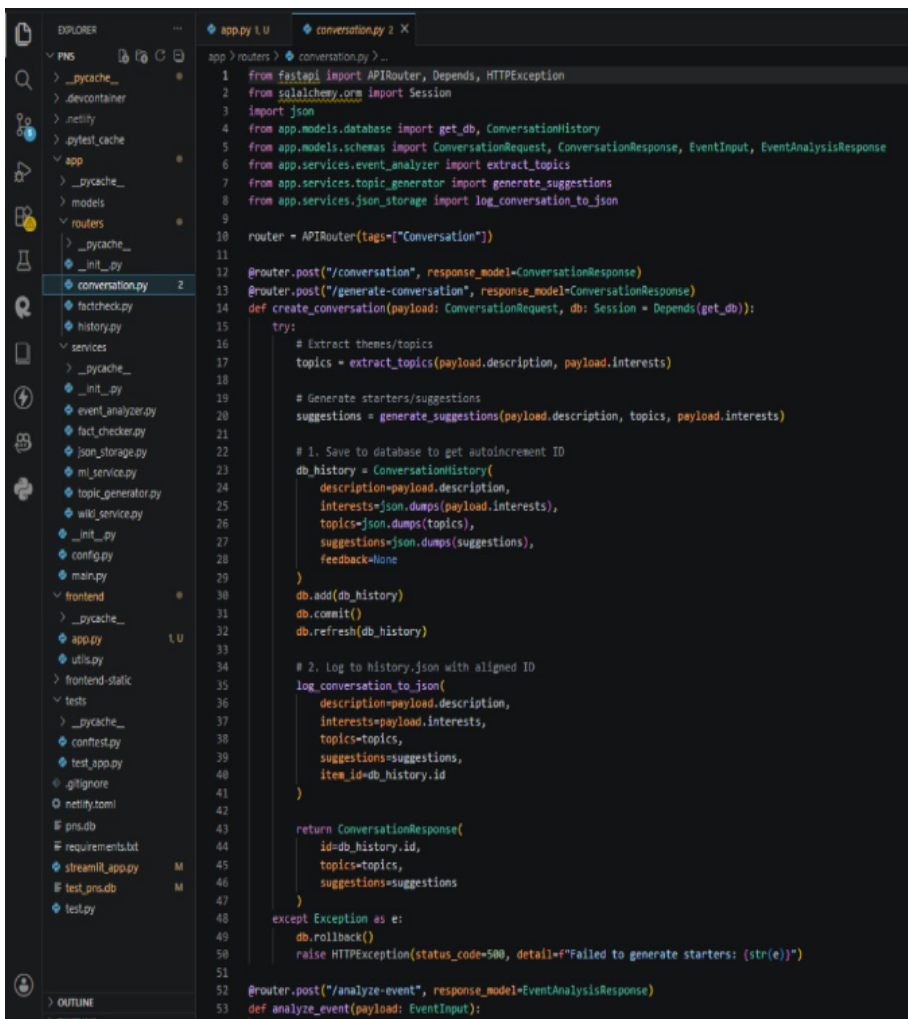
EXPLORER
  app
    __pycache__
    .devcontainer
    .netlify
    .pytest_cache
    app
    __pycache__
    models
    routers
    __pycache__
    __init__.py
    conversation.py
    factchecker.py
    history.py
    services
    __pycache__
    __init__.py
    event_analyzer.py
    fact_checker.py
    json_storage.py
    ml_service.py
    topic_generator.py
    wiki_service.py
    __init__.py
    config.py
    main.py
    frontend
    __pycache__
    app.py
    utils.py
    frontend-static
    tests
    __pycache__
    conftest.py
    test_app.py
    .gitignore
    netlify.toml
    pns.db
    requirements.txt
    streamlit_app.py
    test_pns.db
    test.py
  OUTLINE

app > services > json_storage.py > FEEDBACK_FILE
6 HISTORY_FILE = "history.json"
7 FEEDBACK_FILE = "feedback.json"
8
9 def log_conversation_to_json(
10     description: str,
11     interests: List[str],
12     topics: List[str],
13     suggestions: List[str],
14     item_id: Optional[int] = None
15 ) -> int:
16     """
17     Logs a generated conversation starter event to history.json.
18     Returns a unique integer ID.
19     """
20     history = []
21     if os.path.exists(HISTORY_FILE):
22         try:
23             with open(HISTORY_FILE, "r", encoding="utf-8") as f:
24                 history = json.load(f)
25                 if not isinstance(history, list):
26                     history = []
27         except Exception:
28             history = []
29
30     # Generate a unique sequential ID if not provided
31     new_id = item_id
32     if new_id is None:
33         new_id = 1
34         if history:
35             new_id = max(item.get("id", 0) for item in history) + 1
36
37     new_item = {
38         "id": new_id,
39         "description": description,
40         "interests": interests,
41         "topics": topics,
42         "suggestions": suggestions,
43         "feedback": None,
44         "created_at": datetime.now(timezone.utc).isoformat()
45     }
46
47     # Insert at the beginning so the latest matches are first
48     history.insert(0, new_item)
49
50     with open(HISTORY_FILE, "w", encoding="utf-8") as f:
51         json.dump(history, f, indent=2, ensure_ascii=False)
52
53     return new_id
54
55 def log_feedback_to_json(conversation_id: int, feedback: Optional[bool]) -> None:
56     """
57     Logs feedback action to feedback.json and updates the corresponding
58     entry's feedback value in history.json.

```

Conversation History View

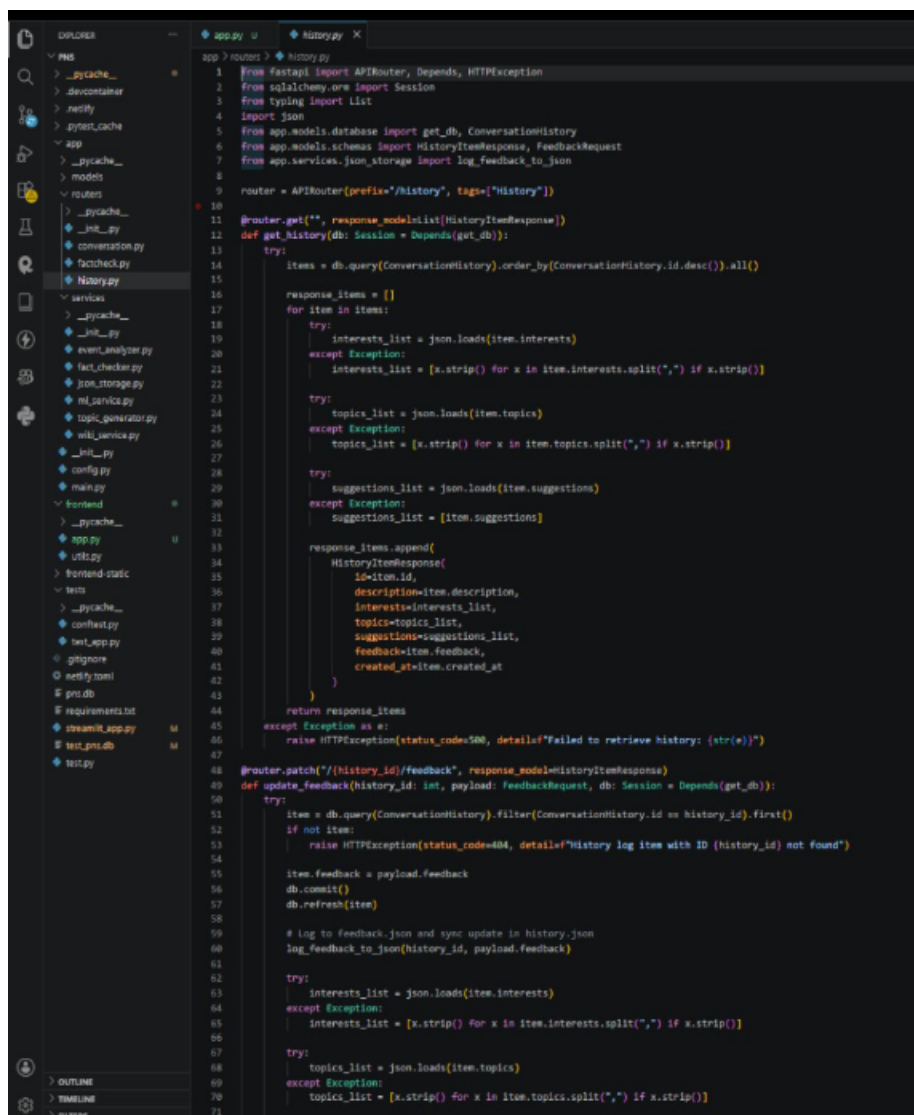
Description : The Conversation History View provides users with a simple interface to review previously generated conversation sessions. It retrieves the stored history from the backend JSON file and displays the most recent conversations in reverse chronological order. Each entry includes the event description, extracted topics, generated conversation starters, and the creation timestamp, allowing users to easily revisit earlier interactions.



```
1 from fastapi import APIRouter, Depends, HTTPException
2 from sqlalchemy.orm import Session
3 import json
4 from app.models.database import get_db, ConversationHistory
5 from app.models.schemas import ConversationRequest, ConversationResponse, EventInput, EventAnalysisResponse
6 from app.services.event_analyzer import extract_topics
7 from app.services.topic_generator import generate_suggestions
8 from app.services.json_storage import log_conversation_to_json
9
10 router = APIRouter(tags=["Conversation"])
11
12 @router.post("/conversation", response_model=ConversationResponse)
13 @router.post("/generate-conversation", response_model=ConversationResponse)
14 def create_conversation(payload: ConversationRequest, db: Session = Depends(get_db)):
15     try:
16         # Extract themes/topics
17         topics = extract_topics(payload.description, payload.interests)
18
19         # Generate starters/suggestions
20         suggestions = generate_suggestions(payload.description, topics, payload.interests)
21
22         # 1. Save to database to get autoincrement ID
23         db_history = ConversationHistory(
24             description=payload.description,
25             interests=json.dumps(payload.interests),
26             topics=json.dumps(topics),
27             suggestions=json.dumps(suggestions),
28             feedback=None
29         )
30         db.add(db_history)
31         db.commit()
32         db.refresh(db_history)
33
34         # 2. Log to history.json with aligned ID
35         log_conversation_to_json(
36             description=payload.description,
37             interests=payload.interests,
38             topics=topics,
39             suggestions=suggestions,
40             item_id=db_history.id
41         )
42
43         return ConversationResponse(
44             id=db_history.id,
45             topics=topics,
46             suggestions=suggestions
47         )
48     except Exception as e:
49         db.rollback()
50         raise HTTPException(status_code=500, detail=f"Failed to generate starters: {str(e)}")
51
52 @router.post("/analyze-event", response_model=EventAnalysisResponse)
53 def analyze_event(payload: EventInput):
```

Feedback History View

Description : The Feedback History View displays all previously submitted user feedback in an organized format. It reads the stored feedback records from the feedback database and presents them with the suggestion text, feedback status (Like/Dislike), and submission timestamp. This feature helps users track their past responses and provides a foundation for future recommendation improvements and system evaluation.



```

1  from fastapi import APIRouter, Depends, HTTPException
2  from sqlalchemy.orm import Session
3  from typing import List
4  import json
5  from app.models.database import get_db, ConversationHistory
6  from app.models.schemas import HistoryItemResponse, FeedbackRequest
7  from app.services.json_storage import log_feedback_to_json
8
9  router = APIRouter(prefix="/history", tags=["History"])
10
11 @router.get("", response_model=List[HistoryItemResponse])
12 def get_history(db: Session = Depends(get_db)):
13     try:
14         items = db.query(ConversationHistory).order_by(ConversationHistory.id.desc()).all()
15
16         response_items = []
17         for item in items:
18             try:
19                 interests_list = json.loads(item.interests)
20             except Exception:
21                 interests_list = [x.strip() for x in item.interests.split(",") if x.strip()]
22
23             try:
24                 topics_list = json.loads(item.topics)
25             except Exception:
26                 topics_list = [x.strip() for x in item.topics.split(",") if x.strip()]
27
28             try:
29                 suggestions_list = json.loads(item.suggestions)
30             except Exception:
31                 suggestions_list = [item.suggestions]
32
33             response_items.append(
34                 HistoryItemResponse(
35                     id=item.id,
36                     description=item.description,
37                     interests=interests_list,
38                     topics=topics_list,
39                     suggestions=suggestions_list,
40                     feedback=item.feedback,
41                     created_at=item.created_at
42                 )
43             )
44         return response_items
45     except Exception as e:
46         raise HTTPException(status_code=500, detail=f"Failed to retrieve history: {str(e)}")
47
48 @router.patch("/{history_id}/feedback", response_model=HistoryItemResponse)
49 def update_feedback(history_id: int, payload: FeedbackRequest, db: Session = Depends(get_db)):
50     try:
51         item = db.query(ConversationHistory).filter(ConversationHistory.id == history_id).first()
52         if not item:
53             raise HTTPException(status_code=404, detail=f"History log item with ID {history_id} not found")
54
55         item.feedback = payload.feedback
56         db.commit()
57         db.refresh(item)
58
59         # log to feedback.json and sync update in history.json
60         log_feedback_to_json(history_id, payload.feedback)
61
62         try:
63             interests_list = json.loads(item.interests)
64         except Exception:
65             interests_list = [x.strip() for x in item.interests.split(",") if x.strip()]
66
67         try:
68             topics_list = json.loads(item.topics)
69         except Exception:
70             topics_list = [x.strip() for x in item.topics.split(",") if x.strip()]
71

```